Lab 10-11

You are given the file `penguins.csv`, which contains information about 344 penguins. The first line of the file represents the name of each attribute documented for the penguins. Each subsequent line in the file describes one penguin, with values for the attributes separated with ‘,’.

The full set of requirements spans the entire document. Please read until the end to understand all you have to do.

1. Create a new file titled `penguins_data.csv` that contains only the following data about the penguins:
    - species: the penguin species (Chinstrap, Adelie, Gentoo)
    - flipper_length_mm: flipper length (mm)
    - culmen_length_mm*: culmen length (mm)
    - culmen_depth_mm*: culmen depth (mm)
    - body_mass_g: body mass (g)
    - island: island name in the Palmer Archipelago
    - sex: penguin sex

*culmen = the upper ridge of a bird's beak.

In this file, save only penguins that have the information filled in for all seven considered fields.

2. Write an application that allows a user to view information about the penguins. The application will have a command-based user interface, allowing the following commands:

> **`print available_data`**

Lists all the .csv files present in a predefined directory. The directory is set before the program is run; the user cannot specify it or change it.

> **`load <filename>`**

Loads data from file with `filename`. The commands below will work on the data loaded from this file.

Example:

```
>load penguins_data.csv
Loaded 344 rows.
```

`> filter <attribute> <value>`

Filters data based on the value for penguin attribute `attribute`, which has to be higher than the given `value` (in the case of numeric fields) or equal to the given `value` (for string fields).

User	Program	Description

User	Program	Description
filter flipper_length_mm 15		User attempts to filter the set of penguins by flipper length.
	The number of penguins that meets the criteria: 280.	The program displays the number of penguins that fit this criteria (in this case, the number of penguins that have a flipper length greater than 15).
	Do you want to save this data to a new file? (y/n)	The user is asked whether they want to save the filtered data to a file.
y		User responds affirmatively. “Y” or “yes” (with any capitalization) will also be considered affirmative answers.
	Please give the filename:	The program asks for the filename where data will be saved
penguins_filtered_flipper15		The user specifies the name of the file to save the data to.
	The data was saved to file: penguins_filtered_flipper15.csv	All data about the penguins that meet the criteria is saved in the file, which should be placed inside the directory with all the available data (the one whose content can be printed through the print available_data) command.
filter island Torgersen		The user attempts to filter the set of penguins by island.
	The number of penguins that meets the criteria: 123.	The program displays the number of penguins that fit this criteria.
	Do you want to save this data to a new file? (y/n)	The user is asked whether they want to save the filtered data to a file.
No		The user specifies they do not want to save the data.
	The data was not saved to file. Displaying it in the console instead: <list of penguins>	The program displays an informative message regarding saving the data to file and prints the information about the penguins that meet the criteria to the console instead.
filter column 13		The user attempts to filter the set of penguins by an inexistent column.
	The specified column does not exist.	An informative error message is displayed.
filter flipper_length_mm abc		The user attempts to filter the set of penguins by an existing column, but specifies a non-numeric value.
	For comparison of numeric attributes, a numeric value must be specified.	An informative error message is displayed.

`> describe <attribute>`

Computes the minimum, maximum and mean value for the given attribute. Displays an error message if the attribute is non-numeric.

Example:

```
>describe flipper_length_mm
Flipper length (mm): min = 32.1 max=59.6 mean=43.9
```

> unique <attribute>

Lists all the unique values for the given attribute and their counts.

Example:

```
>unique species
Adelie: 152 penguins
Gentoo: 124 penguins
Chinstrap: 68 penguins
```

> sort <attribute> <asc | desc>

Print the penguins sorted based on `attribute` in ascending or descending order. You have to implement your own sorting function using one of the following sorting algorithms:

A. Bubble sort
B. Insertion sort
C. Selection sort
D. Quick Sort
E. Merge sort

You may choose any sorting algorithm for which the letter in front can be found in your surname. If none of the letters are in your family name, then take into consideration your first name, as well.

A single sorting function should be implemented, with appropriate parameters (`key`, `reverse` etc).

Measure the execution time and log it into a file that holds the information about all the sorting requests sent to the program.

date_of_run, time_of_run, number_of_rows, sorting_algorithm, execution_time_in_seconds

> augument <percent> <duplicate | create >

The option allows augumentation of the dataset by:

- randomly duplicating entries (penguins) (`duplicate`)
- creating new entries with randomly chosen values from those available in the original set (for string values, choose one from existing values; for numeric values, choose a value between minimum and maximum for that given attribute) (`create`)

with the given percent. Save the data to a new file (the name of which does *not* need to be specified by the user).

Example:

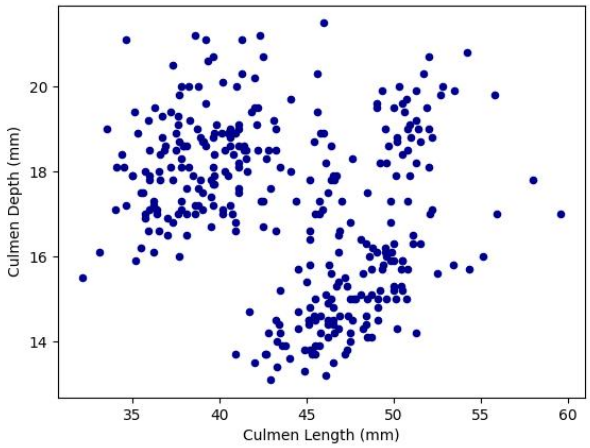
```
>augument 20 duplicate
The dataset was augumented and saved to a new file.

*If the original set contained 200 penguins, now 40 more are added
(entries are duplicated randomly).
```

> scatter <attribute1> <attribute2>

A scatter plot is generated with values for the two attributes. Displays a messaage if either of the two attributes are not numeric. The plot is shown directly or saved to a file for separate visualization - you can choose how you implement.

Example:

```
> scatter culmen_length_mm culmen_depth_mm
```



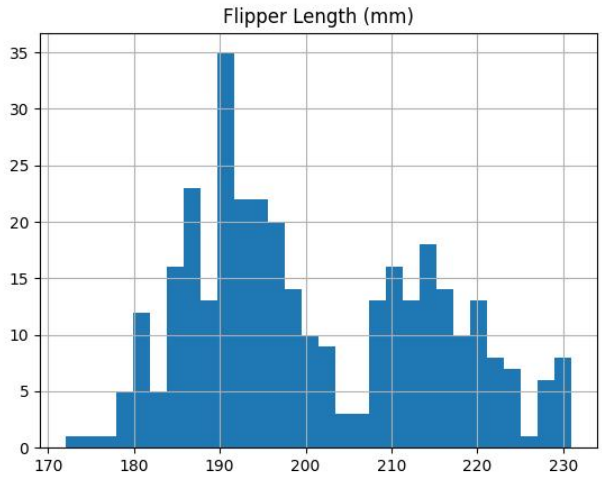
> hist <attribute> <bins>

A histogram with the specified number of `bins` is generated for the given `attribute`. A message is displayed if attribute is non-numeric.

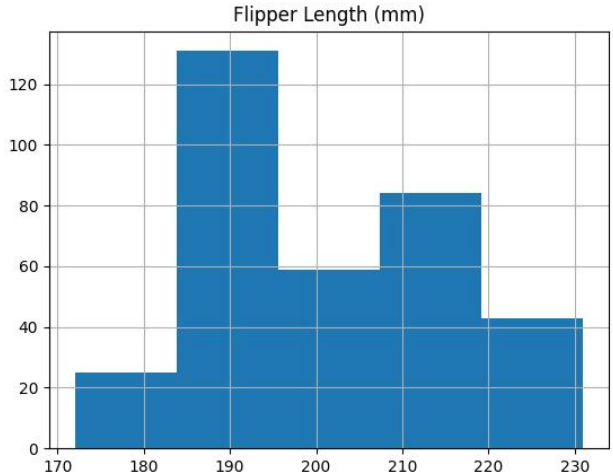
A histogram divides the values within a numerical variable into “bins”, and counts the number of observations that fall into each bin. By visualizing these binned counts in a columnar fashion, we can obtain a very immediate and intuitive sense of the distribution of values within a variable.

Example:

```
> hist flipper_length_mm 30
```




```
> hist flipper_length_mm 5
```

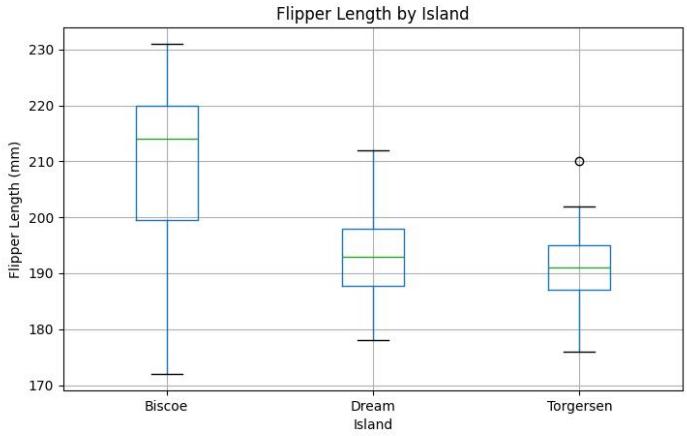


> boxplot <island | species > <attribute>

Create a boxplot for the specified attribute with regard to either island or species.

Example:

```
> boxplot island flipper_length_mm
```



`> help`

Print all available commands and short indications regarding their use.

`> quit`

Exit the program.

Other requirements (mandatory, but they are not commands in themselves):

- For filter, unique and describe, **specify the time and space complexity of all the functions used to implement the action.**
- **Implement tests** using unittest for the filter, unique and describe functionalities.
- Implement your own exception classes.
- For sorting, **have at least one run for each of the following dataset sizes documented in the sorting log file** (use the augument command to create bigger datasets).
    - Less than 100 penguins
    - Between 500 and 1000 penguins
    - Between 1000 and 2000 penguins
    - Between 5000 and 10000 penguins
    - More than 10000 penguins

Observations:

- You can choose how to split your effort between Week 11 and Week 12.
- Lab 13 will require you to add some functionality to the application you've built during these 2 weeks (keep this in mind when you implement).
- You can use classes, but it is not mandatory.
- Separate UI, file reading and writing operations, and other operations (at least).
- Do not use pandas/dataframes.
- For plots, you can use any library you want (examples: `matplotlib, seaborn`).

Bonus possibilities:

For the last labs, you will not get an additional requirement during the lab, but you can implement any (or all) of the following to obtain additional points:

1. **[0.3p]** Add a GUI to the application (`tkinter, PyQt`). Every action corresponding to a command should be available in the GUI, but it is not mandatory for the plots to be visualized in the GUI, it is enough that the user generates the plot based on a given selection (the plot is saved to a file which can be visualized separately).

Deadline: Week 14

2.
`>classify culmen_length culmen_depth flipper_length k`

[0.2p] Add a simple classifier to your application. This should allow the classification of a penguin into a species based on `culmen_length`, `culmen_depth_mm` and `flipper_length_mm`. Use the k-Nearest neighbors algorithm.

Indications:
https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/
https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm

Example:

```
> classify 45.2 17.1 200 5
Chinstrap penguin
```

Deadline: Week 14

3. **[0.12p]** Implement all sorting algorithms (A-E) and run them, logging the performance for the five cases discussed in "other requirements" across five runs each. Add a command that prints a report regarding the average time taken by each algorithm for each dataset size category.

Deadline: Week 13

4. **[0.03p]**

`>random_fact`

Display a random fact about penguins, chosen from a predefined list of at least 15 facts.

Deadline: Week 12

5. [0.01p]

`>draw_penguin`

Draw an ASCII penguin.

Deadline: Week 12